F1 Telemetry Design Document

Introduction:

For this project I want to make a dashboard for formula one data whether that is historical data or live data. F1 is a complicated sport at times and it can be very hard to keep track of what is going on over a race, especially if the live feed is not showing you the information relevant to you and who you are rooting for. The intention for this dashboard is to be easily accessible for regular views but delve as deep into the data as you would want, if at all possible customisability would be a huge step in that directions and have a modular dashboard.

Key Functionality:

- Live mini map showing all drivers position on track
- Timing tower showing current positions, times and gaps
- Strategy breakdown showing current tyre and how long a driver has been on them
- Historical Data (Show results from that weekend in the past and standings at that time)

Potential Functionality:

- Team radios
- Mini sectors (small segments within a sector)
- Live driver and constructor standings
- Weather breakdown
- Compatibility with most devices (PC and Mobile)

Open F1 API:

The open F1 API allows historical and live data streaming to be accessed at any time, this can be used for Python or JavaScript. The main use-cases for this API are either a Web-app or local application and both of which have their own pros and cons. For this project I will be making a web app for a few reasons. Firstly updating the app for every use is a lot easier, the current version will be pushed to the web page and there is minimal friction for the user to worry about. Secondly it is easier for compatibility between devices, since it will be a web page the only thing I need to worry about is making sure the styling is right across devices, for a local app it will need to have a bespoke executable for each supported platform and porting it to the other devices.

For historical data the api uses a REST api, this means it acts like a snapshot in a moment in time. For this use case the snapshot will be an end to a session which will hold all the data for that session in the API URL call. This should not be used for live data because it is wildly inefficient since each time data is called it will return the whole dataset each time causing massive inefficiency which is unecessary since the OpenF1 API supports live data streaming natively.

For Live data MQTT or Websockets is used which will stream data live directly which is a lot lighter than the REST API and also provides the data as and when is is available with minimal delay. Both methods of streaming live data produce the same data, at the same speed, the only difference is the delivery. MQTT connects via raw TCP which is to be used in backend/server environments, Websockets is MQTT tunnelled through WSS and used for browser projects. The end result is the same.

Tech Stack:

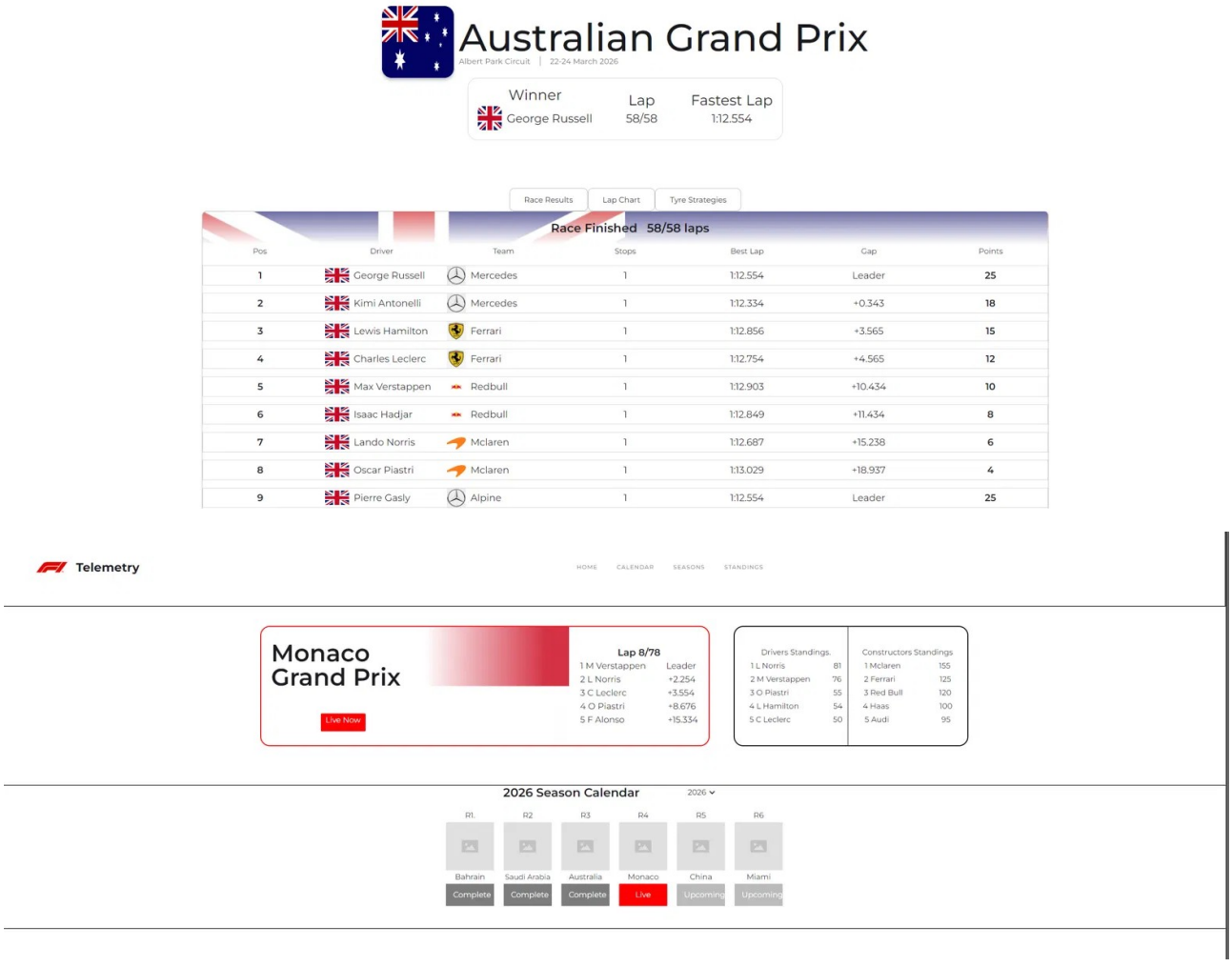
- HTML, CSS, JS
- Github
- Potential Backend server

For this project I will be keeping things simple for myself and not overcomplicate it, this is because of the nature of the project being my first time running via an API I don't want to overextend myself. I would like to use REACT but I am not familiar with it, if things go smoothly adding retroactively could be an option.

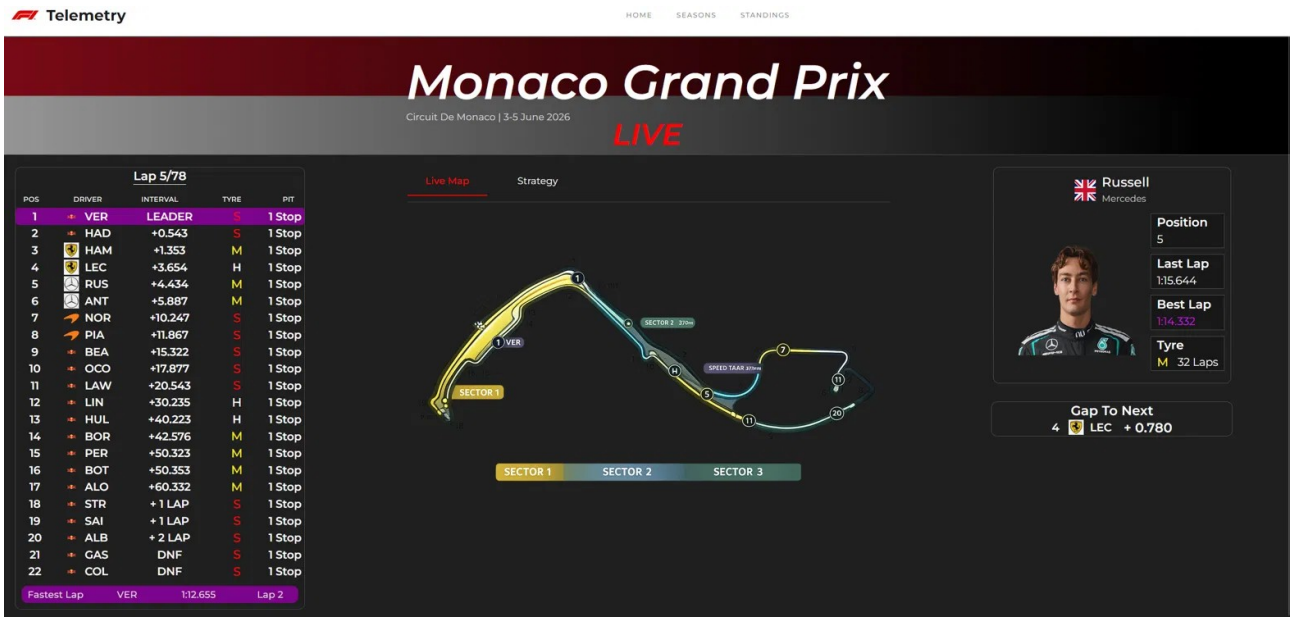
This project will utilise github for source control to ensure the projects files are synced across workstations properly as well as a backup in case the local build goes wrong for whatever reason.

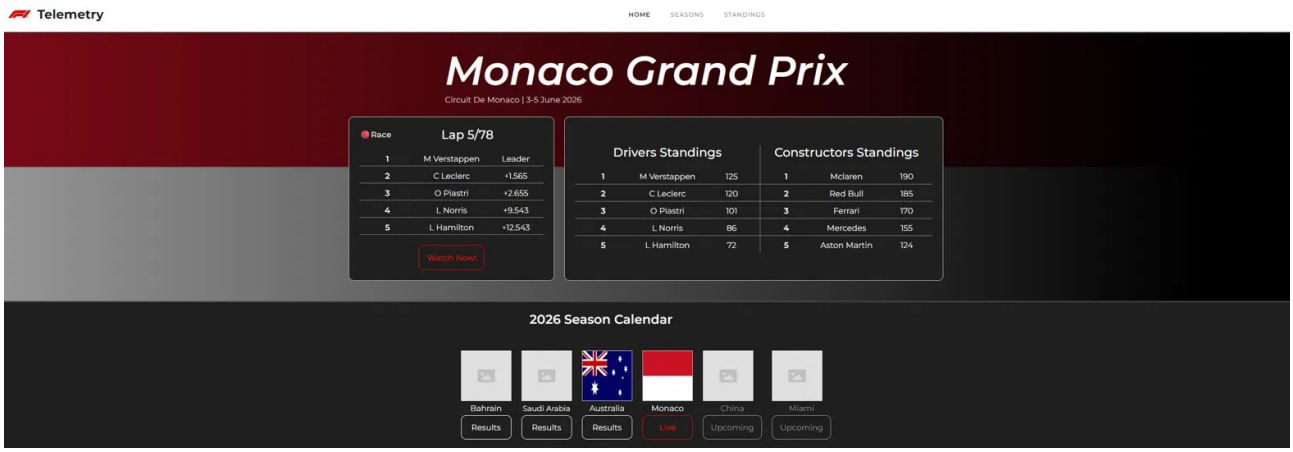
There is potential for a backend server to be added to the project but once again I don't want to overcomplicate it, this server would act as a middle man holding the credentials for the API usage. This would move the credentials off the local computer and if this were to be a released project a backend of some kind would be essential so the API cannot be hijacked or abused, the server would also hold the current valid token so the user cannot manipulate the token in anyway.

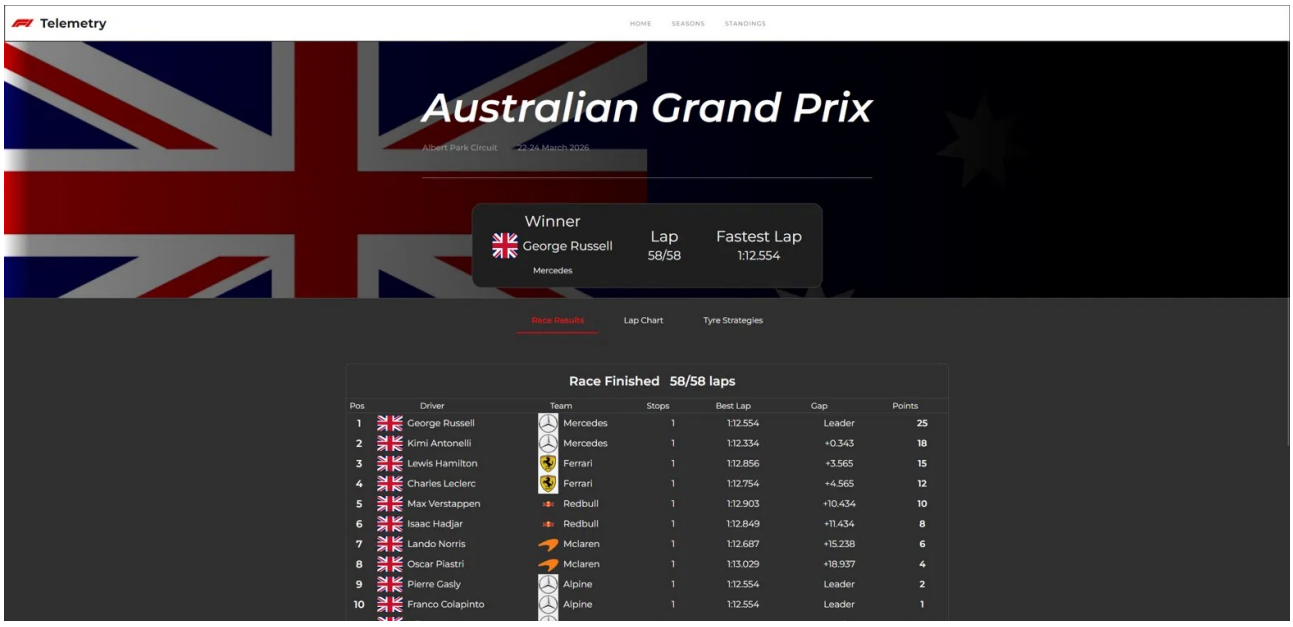
Initial Design:



Final Design:







Week 1 & 2:

In these first couple week I have really spent my time nailing down a design language for this site, the goal for this site is to be visually engaging, with good data breakdowns geared towards the average viewer which means the data shown will be easily digestible and not super crowded. Design is very important with this, in my research I have found multiple sites that have a similar functionality but the either the flow of the site is very confusing or the screen is overloaded with data screens and can get lost if you don't know what it means. That being said I also don't want it to be limited to casual fans and allow the technical fans to get into the nitty gritty. This version of the site will aim closer to casual fans focusing on good quality easily readable data vs deep and dense but I want to leave myself open to allowing deeper analysis.

I used a website development tool to visualise my ideas and slowly iterate in an environment that allowed it with a bit more ease than tinkering in code as well as this, it allowed me to test the flow of the site and nail down the experience. The flow of the site will follow this skeleton as I have given the test site to several people which said the flow made sense and is as follows:

Home page - If there is a live event the first box will show current positions for the top 5 and lap with a button to take the user to the live data page, if there is no live event it will show the top 5 of the previous race. Below this hero section is a calendar of the current season and allows the user to view results of the previous races and show the upcoming races. Next to the live/previous race will be the top 5 in the drivers standings and the top 5 in the constructors standings updated after each race.

Live page - This will show the live data of a current session showing a minimap and each car on the track, a timing tower and the ability to click on any driver and show their data and gaps of the cars either side

Session page - This is the page you see when you click on the button under a race on the calendar on the home page, this will show you each session from a selected race weekend as well as the top 3 of each session. This will then allow the user to click each session and see the deeper results on the Result Page showing the full standings and any additional data there.

Results Page - This will give the results of any given session in a race weekend showing full post session standings and any data related to that session.

And 2 pages I have no mapped out due to limitations of the software are the seasons page and the standings page

Season Page - This will give the users the option to go back in time and view results from a season. This page will show the calendar and the ability to select specific seasons from 2023 - current (due to limitations of the openF1 api only going back to 2023).

Standings Page -This is the most simple page in the site, this will simply show the full drivers and constructors standings.

Week 3:

This week saw the beginning of programming the home page, at this stage it was mostly HTML making the skeleton of the site with a rough idea of what was going where. I was following the plan I had outlined previously via webflow so I knew what I was working towards. There was a little bit of CSS work too but nothing too intense

Week 4:

This week was continuing work on the home page of the site, this was mostly a lot more CSS work working on the style of the site since the HTML side was pretty much finished aside from a few tweaks where necessary for the site flow. The site now looks like what I had planned but has zero functionality at this stage. I have also asked people around me for feedback on the site and what they feel when they see the site and if it is something they would use if they clicked on it.

Week 5:

This week consisted of finishing off the CSS work and beginning the java script. Since this is my first time working with an API within a website so I am expecting this to be tricky and a fun challenge. The first thing to fill in is the hero banner with the key information such as event name, drivers standings and the winner to the previous race. This has taken longer than I expected due to the nature of not knowing what I am doing or have a workflow sorted out yet.

I first made a JS script that I could use to simply call the information universally for historic data and as such can be used everywhere without copy and pasting the same method everywhere to process it. Once this was sorted I spent time carefully combing through the API documentation to find what information I needed to call. I started off with the session endpoint but I evidently did not look through the documentation well enough since/didn’t interpret it well enough as I ended up with a bit of a messy script since I was calling information from many endpoints that were not necessary. When I took a closer look I found the meeting endpoint which had the majority of the information needed for the hero banner such as country flags, event name (Australian Grand Prix), dates and more. Prior I was extensively processing information which was not necessary at all.

Week 6:

This week was continuing the hero banner, last week I had finished the showing the flag of the race and the race name after some refactoring. This week I built the standings for the hero banner. This, after the experience last week, was fairly simple by making html rows and a grid layout in the container to shape it. I used a foreach loop to only show the first 5

results. I had to work around a little bit as the driver standings endpoint shows driver numbers rather than driver names so I have to combine it with the driver endpoint and for each entry in the standings return the driver name with the same driver number (This is not an issue because no driver at one time will have the same number).

Since this less time than expected I started the calendar carousel, I setup the skeleton for it with placeholder values before making the js code to fill it in to make the functionality work. I want this page to be scalable to work on different size displays and press a button either side of it to either go to the next entry or previous entry. This proved to be quite challenging and any tutorial I found did not work in the way I intended or wanted it to with the majority moving by a whole page rather than an increment of 1. For this I worked with Claude to help me develop this and explain how it works and ended up being far more simple than any tutorial, I'm sure it is not perfect but for now to get something that works and is deliverable it is fine.

Week 7:

This week was a bit of a tidy up week, I have done a lot of work recently but there was one glaring issue with the site, it was extremely slow and was very frustrating to use. There were 2 issues, firstly there are many async methods running sequentially one after another, secondly none of the information is saved locally. Both of these combined means the data being called is very slow, not at once and calling duplicate data every time you load the site. This makes testing changes very difficult. I am being careful too as I don't want to spend too much time optimising the letting features fall to the wayside before submission. As such I implemented some fixes for the most intensive calls so every method is not calling the Api itself, centralise some methods and store locally the calendar and the years.

After this I laid the ground work for the next page being the results summary to show the results for each session in a selected race weekend via the calendar at the bottom of the home page and also implemented the idea for a dropdown menu to change the year of the calendar, at this stage I am unsure if I am going to keep it move it to a dedicated calendar page on the site to show the historic years and only the current year on home page or not.

Week 8:

This week was a slightly slower week but I began work on the results summary page where the user clicks a button on the calendar on the home page to show the summary of the race weekend showing the session name and the top 3 of the session. On this page the user will be able to decide which session to view and get in depth information on a given session. The plan is for the user to be able to see a replay of the session as if it was live but this is up in the air as time is starting to become a concern to get everything ready for submission.